Computer Vision: Assignment 3
Deep Learning for Computer Vision

Pablo Mesejo and Siham Tabik
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



Index

- Submission rules
- Assignment description
- Brief introduction to fastai

Index

- **Submission rules**
- Assignment description
- Brief introduction to fastai

Submission rules

- Only *.ipynb* files should be submitted (including code, results, analysis and discussion).
  - Don’t upload the images!
- The submitted notebook must not write files to Google Drive or create unnecessary output files when executed.
- The structure of the templates must be respected.

Submission

- Deadline: 20 December
  - Please, plan well in advance, and make responsible/rational use of Google Colab.
    - Having Google Colab Pro does not imply infinite resources!
- Maximum score: 20 points
- Submission Site: https://pradogrado2627.ugr.es/

- **Explanation/discussion accompanying code and results is essential.**

Goals

- Learn how to implement **convolutional neural networks using fastai/PyTorch**.
- Understand the concepts of **feature extraction, transfer learning** and **fine-tuning**.
- Begin to familiarize yourself with **explainable AI**.

- This is an assignment oriented towards **image classification, regression and segmentation using Deep Learning**.

Materials at your disposal

- This **introduction to assignment 3 and fastai**
- Templates:

  P3_Ejs1y2_template.ipynb

  P3_Ej3_template.ipynb

  P3_Ej4_template.ipynb
- A help guide with codes and examples:

  P3_HG.ipynb

Index

- Submission rules
- **Assignment description**
- Brief introduction to fastai

Exercise 1: BaseNet in CIFAR100 (5 points)

1. Create a simple model (called BaseNet)

2. Train and validate it with the CIFAR100 dataset

Superclass	Classes
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchids, poppies, roses, sunflowers, tulips
food containers	bottles, bowls, cans, cups, plates
fruit and vegetables	apples, mushrooms, oranges, pears, sweet peppers
household electrical devices	clock, computer keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium-sized mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple, oak, palm, pine, willow
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn-mower, rocket, streetcar, tank, tractor

60,000 images
100 classes



https://www.cs.toronto.edu/~kriz/cifar.html

Exercise 1: BaseNet in CIFAR100 (5 points)

Layer Type	Kernel Size (for convolutional layers)	Input | Output dimension	Input | Output channels (for convolutional layers)
Conv	11x11	32x32 | 32x32	3 | 5
Sigmoid	-	32x32 | 32x32	-
MaxPooling	2x2	32x32 | 16x16	-
Conv	7x7	16x16 | 16x16	??? | 10
Tanh	-	16x16 | 16x16	-
AvgPooling	2x2	16x16 | 8x8	-
Conv	3x3	8x8 | 6x6	??? | 15
Sigmoid	-	6x6 | 6x6	-
FC	-	??? | 250	-
ReLU	-	250 | 250	-
FC	-	250 | 125	-
ReLU	-	125 | 125	-
FC	-	125 | ???	-

Architecture you have to implement in fastai

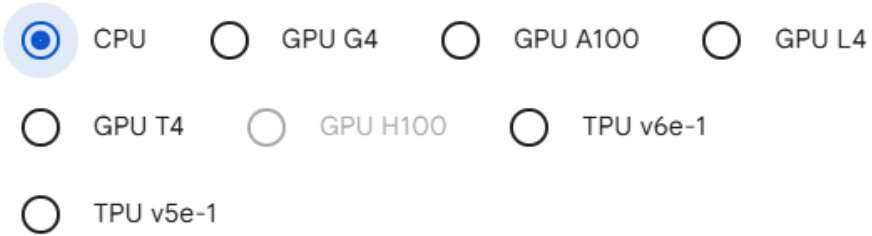
Taking into account that this is a multiclass classification problem, what is the most natural/common choice for the activation function and loss function?



What are the values that correspond to ????

Exercise 1: BaseNet in CIFAR100 (5 points)

3. Compare training in CPU vs GPU



- In case you want to learn more about GPUs:
  - https://jax-ml.github.io/scaling-book/gpus/
  - https://sebastianraschka.com/teaching/pytorch-1h/
  - https://www.jeremyjordan.me/distributed-training/

Exercise 1: BaseNet in CIFAR100 (5 points)

4. FC layers force the input to have always the same size & add a lot of weights.
   - **How can we remove the FC layers from our model? Implement a new BaseNet model without FC layers**



Exercise 2a: Improvement of the BaseNet model

(4 points)

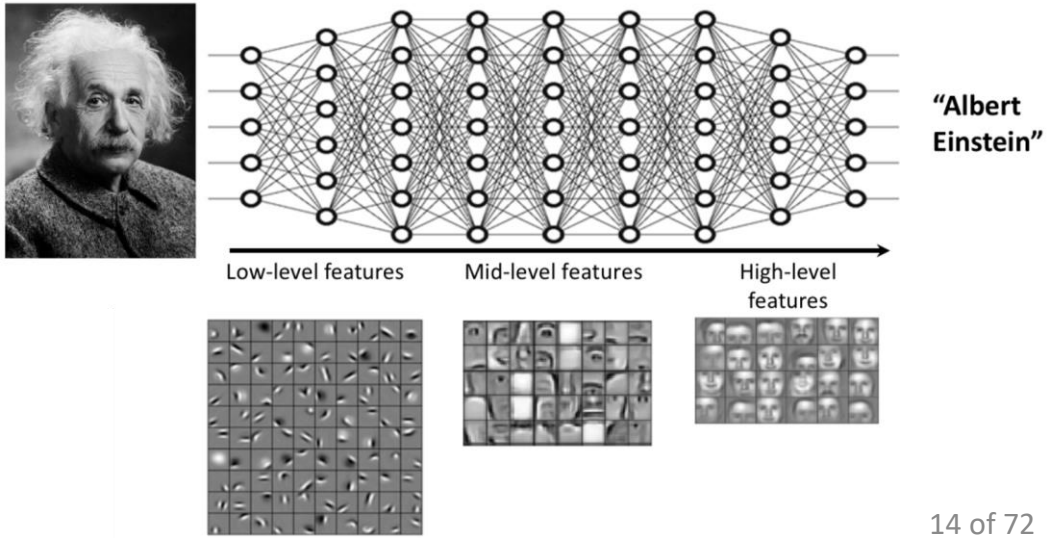
- Once you have implemented and validated BaseNet, you should improve the network by means of those alternatives that you judge to be appropriate:
  - Data augmentation
  - Network depth augmentation
  - Batch normalization
  - Regularization
    - Dropout
    - Early-Stopping
  - Others?
    - You can try things you have seen in theory or from other sources.
    - Unleash your creativity and intuition.
    - Innovation, complexity and good use of PyTorch/fastai will be highly valued.

Validation accuracy is expected to reach at least 65%.

- Always remember to **justify your decisions** (it is not about testing for the sake of testing) and to **clearly show the final architecture**.

Exercise 2b: Visualization and interpretation of the learned representations (1 point)

- Visualize at least 8 RGB filters from the first convolutional layer.
- Select an image from the validation set and visualize at least 4 activation maps at three different depths.



Zeiler, M. D., & Fergus, R. (2014). Visualizing and Understanding Convolutional Networks. ECCV. https://arxiv.org/abs/1311.2901

Yosinski, J., et al. (2015). Understanding Neural Networks Through Deep Visualization. ICML Deep Learning Workshop. https://arxiv.org/abs/1506.06579

Olah, C., Mordvintsev, A., & Schubert, L. (2017). Feature Visualization. Distill. https://distill.pub/2017/feature-visualization/

Stanford CS231n. Visualizing What ConvNets Learn. https://cs231n.github.io/understanding-cnn/

Exercise 2b: Visualization and interpretation of the learned representations (1 point)

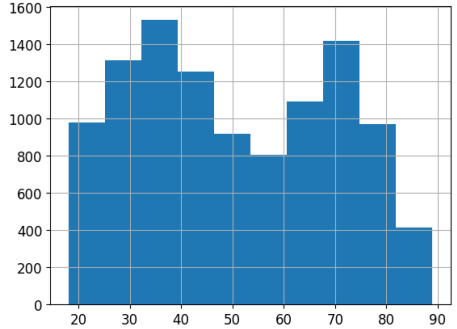
- Visualize at least 8 RGB filters from the first convolutional layer.
- Select an image from the validation set and visualize at least 4 activation maps at three different depths.

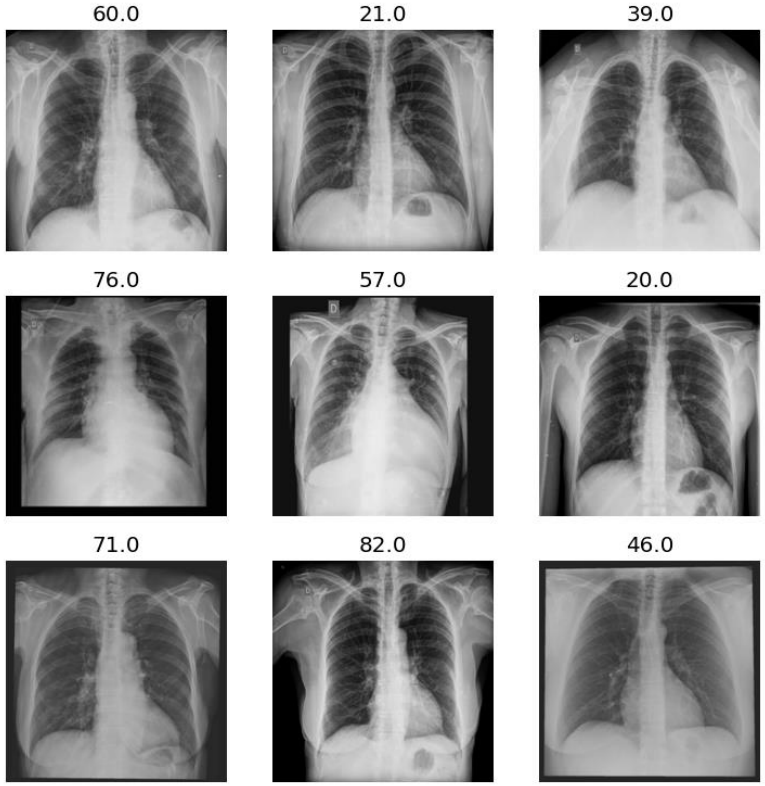
Useful PyTorch concepts

- Convolutional filters: **`Conv2d.weight`**.
- Activation maps: output tensors of intermediate layers.
- Forward hooks: **`register_forward_hook()`** to capture intermediate activations.
- Remember to distinguish model weights from image-dependent activations.

Exercise 3: Model transfer and fine-tuning with ResNet50 for the SPR X-Ray Age Prediction Challenge (5 points).

Age varies from 18.0 to 89.0



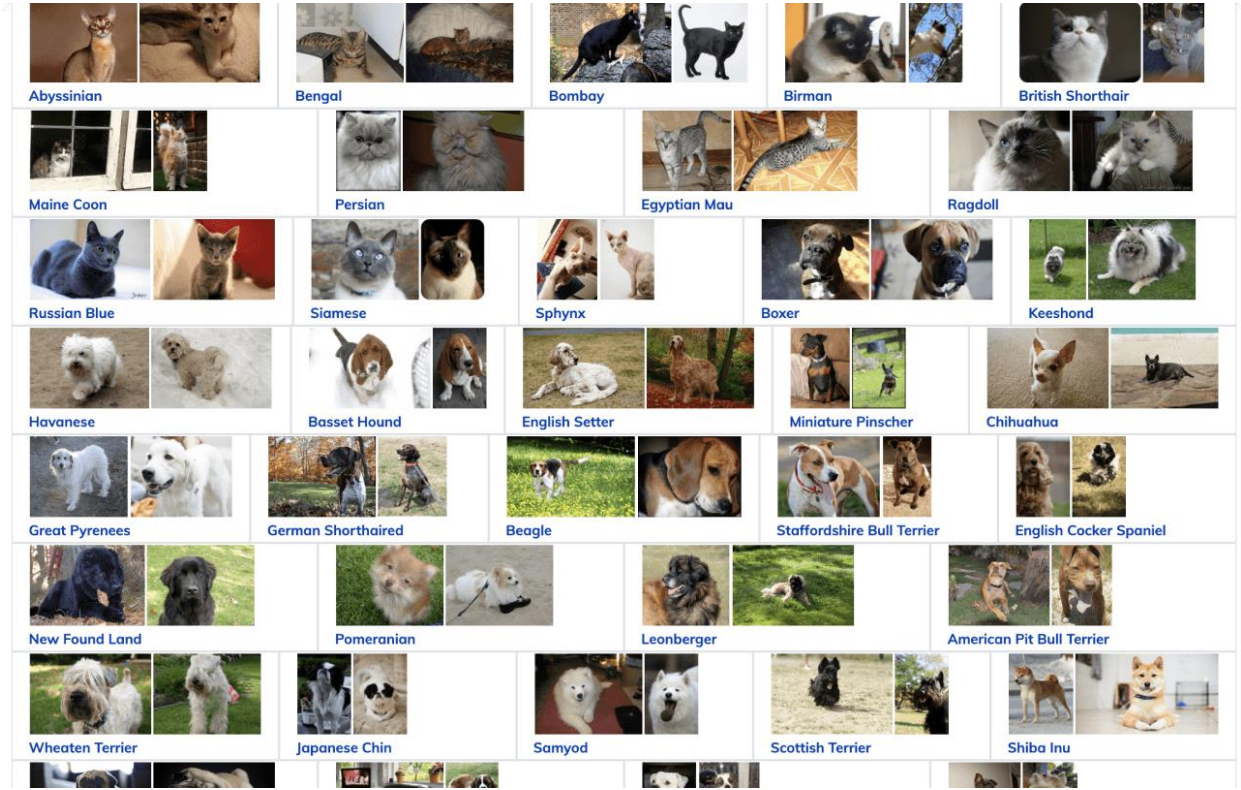


We start from a **model trained in a classification problem**, and we try to **apply it in a regression problem**.

Exercise 3: Model transfer and fine-tuning with ResNet50 for the SPR X-Ray Age Prediction Challenge (5 points).

1. **Train from scratch the entire ResNet50.**
2. **Use ResNet50 as a feature extractor**.
   - Remove the final fully-connected (FC) layer of ResNet50, replace it by a FC layer of the dimensionality of the new problem, and **train the new weights of this FC layer** (while keeping frozen the remaining weights in the network).
   - **Compare with the head introduced by default by fastai**.
3. **Fine-tune the entire ResNet50.**
4. **Fine-tune the entire ResNet50, but posing the task as a 72-class classification problem (age classes $\in \{18, \dots, 89\}$).**
5. **Design and evaluation of an alternative architecture for regression.**
   - We want to study whether a smaller architecture, with a head specifically designed for this task, can **achieve competitive performance compared to ResNet50** while using fewer parameters and computational resources.
6. **General conclusions based on all previous experiments.**

Exercise 4: DenseNet121 fine-tuning, explainable AI and segmentation in the Oxford-IIIT pet dataset (5 points)



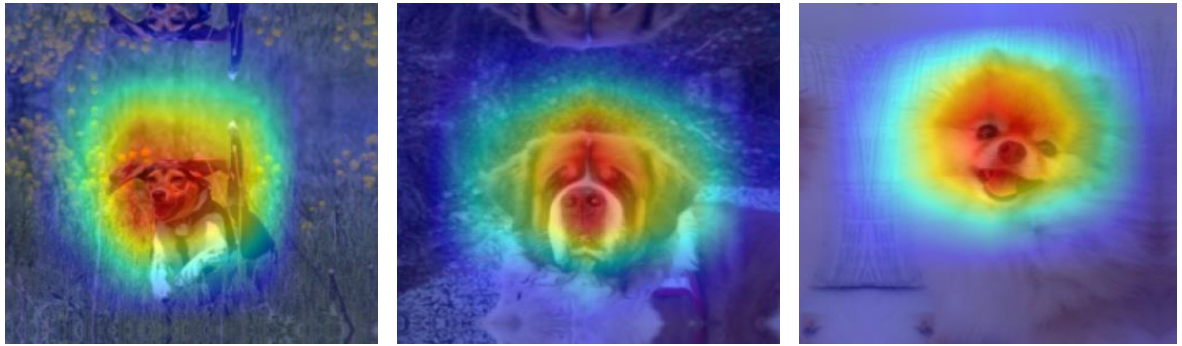
3,680 images for training and validation
3,669 images for test

37 categories

Exercise 4a: DenseNet121 fine-tuning and explainable AI in the Oxford-IIIT pet dataset (2 points).

- We have to **fine-tune DenseNet121 on this classification problem**.
- We **use Grad-CAM to visualize input image regions that are relevant for the prediction**, and we compare Grad-CAM maps and actual segmentation ground truth masks.
- Two conceptual questions: **DenseNet121 description** + **explainable AI limitations**.

Very important references:
https://jacobgil.github.io/pytorch-gradcam-book/introduction.html
https://github.com/jacobgil/pytorch-grad-cam



Selvaraju, Ramprasaath R., et al. "Grad-cam: Visual explanations from deep networks via gradient-based localization." Proceedings of the IEEE international conference on computer vision. 2017 (https://arxiv.org/pdf/1610.02391.pdf)

Exercise 4b: U-Net fine-tuning to segment the Oxford-IIIT pet dataset (3 points).

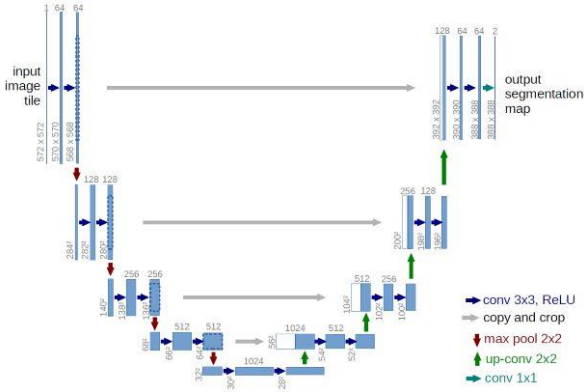
Classification vs. Semantic Segmentation

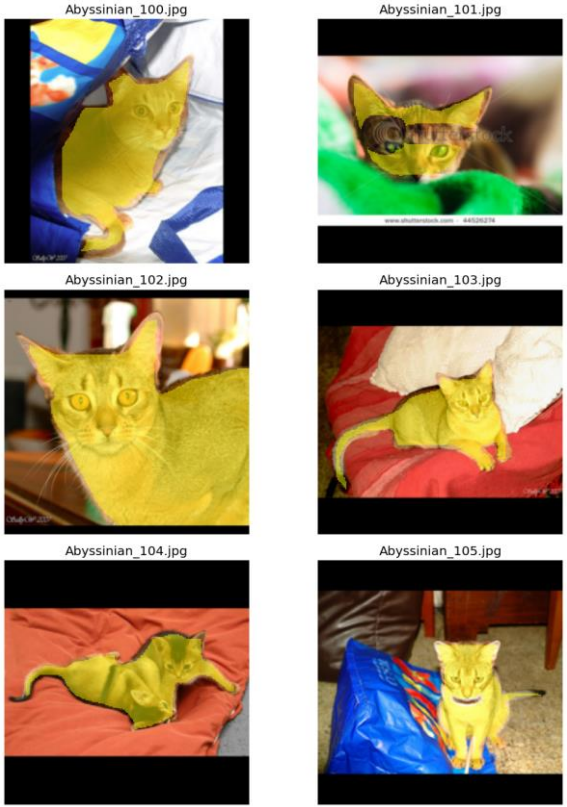
- Image classification: one class label per image.
- Semantic segmentation: one class label per pixel.

Our approach

- U-Net with a pretrained ResNet18 encoder.
- Encoder: feature extraction and spatial downsampling.
- Decoder: spatial upsampling, using skip connections.
- Output: binary segmentation mask (pet vs. background).

unet_learner:
https://docs.fast.ai/vision.learner.html#unet_learner

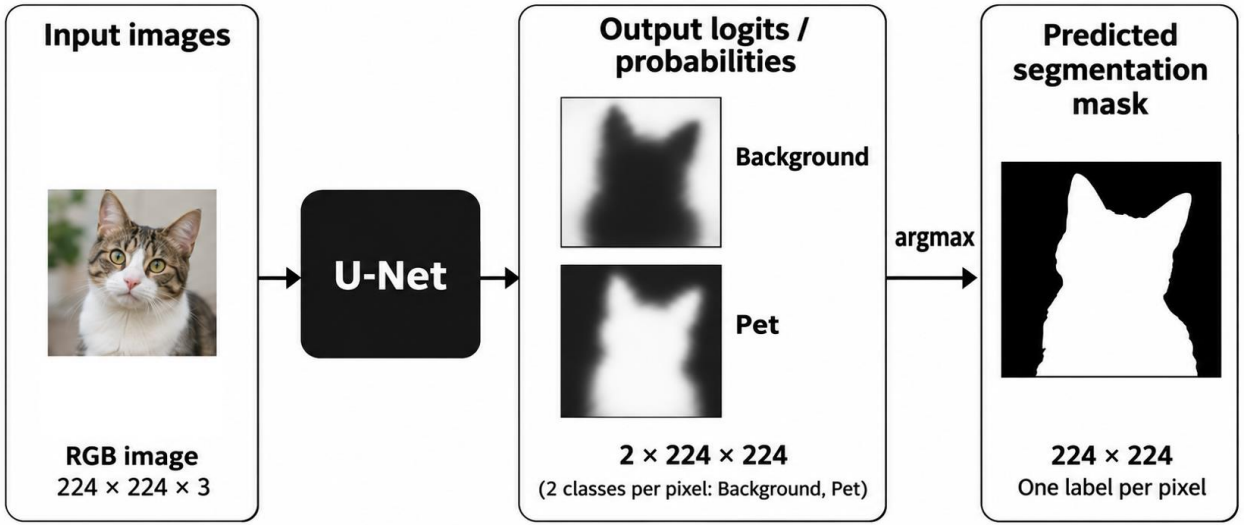




Exercise 4b: U-Net fine-tuning to segment the Oxford-IIIT pet dataset (3 points).

Segmentation mask labels:
0: Background.
1: Pet.
255: Uncertain boundary (ground truth only).

Important:
The U-Net predicts only two classes (background or pet). Pixels labeled 255 in the ground truth are excluded from both the loss function and evaluation metrics, regardless of the predicted class.

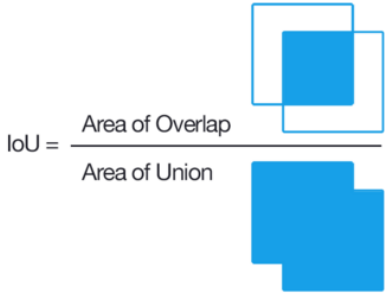


Pipeline: RGB image (224 × 224 × 3) → U-Net → two-class per-pixel predictions (2 × 224 × 224: Background, Pet) → argmax → final segmentation mask (224 × 224), with one label per pixel.

Figure created with the assistance of ChatGPT (GPT-6).

Exercise 4b: U-Net fine-tuning to segment the Oxford-IIIT pet dataset (3 points).

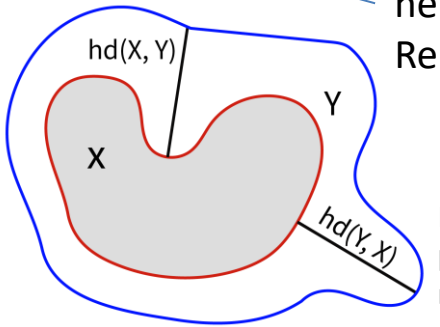
IoU and Dice measure the overlap between predicted and ground-truth regions.



Evaluation:

- IoU (Jaccard Index).
- Dice Similarity Coefficient.
- Hausdorff distance.

Hausdorff distance measures the largest nearest-neighbour distance between two sets of boundary points. Relevant tools: [scipy.ndimage] and [scipy.spatial.distance].



It measures the maximum distance between the boundary of a predicted segmentation mask and the boundary of the ground truth mask. Because it calculates the maximum error, the standard Hausdorff distance is highly sensitive to isolated false positives (outliers).

Index

- Submission rules
- Assignment description
- **Brief introduction to fastai**

Highly Recommended References

- **Book with Colab Notebooks: https://github.com/fastai/fastbook**

- Course "Practical Deep Learning for Coders 2022" (https://www.youtube.com/playlist?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU; https://course.fast.ai/; academic year 2019: https://course19.fast.ai/videos/).

- Jupyter Notebooks by Jeremy Howard: https://www.kaggle.com/jhoward/code

Founding researcher (fast.ai), Distinguished Research Scientist (University of San Francisco), former President and Chief Scientist (Kaggle)

Highly Recommended References

- **Book with Colab Notebooks: https://github.com/fastai/fastbook**

- Course "Practical Deep Learning for Coders 2022" (https://www.youtube.com/playlist?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU; https://course.fast.ai/; academic year 2019: https://course19.fast.ai/videos/).

Note on these materials: The strategy of the book (notebooks and videos) is *top-down*: you start from the code, experiment with it, extract intuitions, and then analyze how it works and go deeper into the fundamentals.

Our framework: fastai

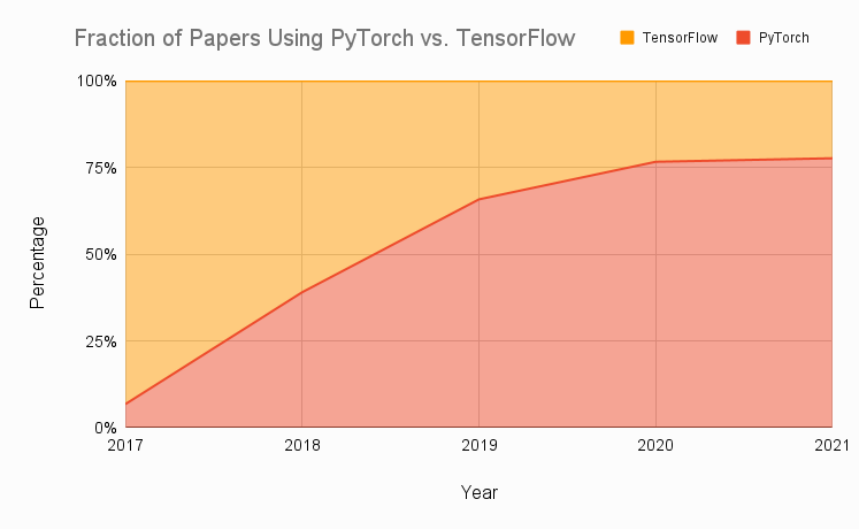
We use **fastai** (based on **PyTorch**).
Main reference: https://docs.fast.ai/

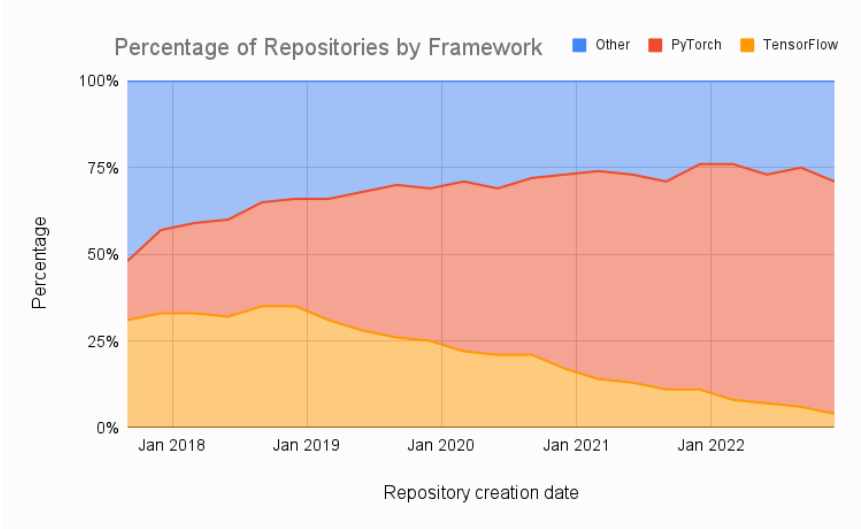
PyTorch Cheat Sheet: https://www.mad.tf.fau.de/files/2019/07/pytorch-cheatsheet-en.pdf
PyTorch documentation: https://docs.pytorch.org/tutorials/index.html

	Keras (TensorFlow API)	fastai (PyTorch API)
Strengths	• Very simple	• Uses PyTorch (the most popular DL tool today)
Weaknesses	• Possibly, less complete and flexible than fastai/PyTorch	• Possibly, longer learning curve • It has so many high-level functionalities, that you may not fully understand what is being done at lower level (e.g. test data normalization)

Our framework: fastai

We use **fastai** (based on **PyTorch**).
Main reference: https://docs.fast.ai/





https://www.assemblyai.com/blog/pytorch-vs-tensorflow-in-2023/

Our framework: fastai

We use **fastai** (based on **PyTorch**).
Main reference: https://docs.fast.ai/

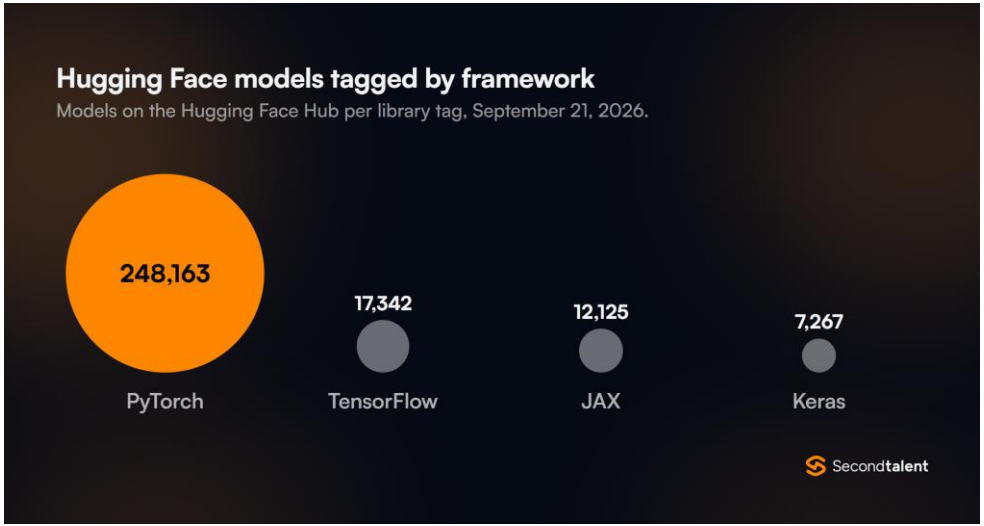
When using fastai you have access to the full potential of PyTorch and new features. As a result, you do not have to write so much code.

But, if you prefer to code using plain PyTorch, feel free to do it!!!

Example using AdamW's step in PyTorch vs fastai:
https://youtu.be/8SF_h3xF3cE?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU&t=1903

Why PyTorch is more popular than TensorFlow nowadays?

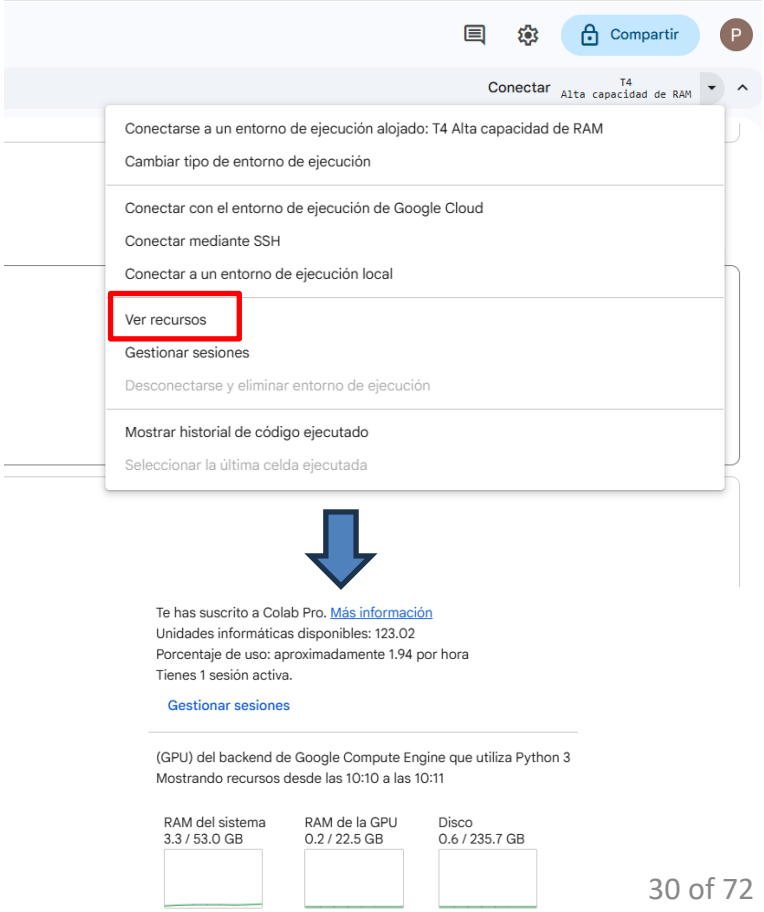
- Ease of use ("you can do anything that PyTorch does in TensorFlow. It will just take you twice as much effort to write the code.")
- PyTorch has more models available:



PyTorch vs TensorFlow: Usage, Popularity and Performance in 2026 - Second Talent

Google Colab: Compute Units

- **Monthly renewal.** Compute units are replenished each month.
- **If units run out.** High-performance GPUs and high-memory runtimes become unavailable. Standard CPU runtimes remain available under free-tier limitations.
- **Consumption example.** At 1.27 units/hour, 30.66 units provide approximately 24 hours of runtime with that same GPU configuration.



Good Practices for Efficient Colab Usage

- **Disconnect idle runtimes.** GPU compute units are charged while a GPU runtime is active, even if no code is executing.

  `Runtime → Disconnect and delete runtime`

- **Use CPU for preparation.** Debug code and clean data (if possible) on CPU; switch to a GPU only when needed for training or inference.
- **Save checkpoints regularly.** Back up model weights to Google Drive so interruptions do not erase training progress.
  - **During development**: You may save checkpoints and other temporary files as needed.
  - **Submission**: Disable or comment out all instructions that save files or checkpoints. The submitted notebook must not write files to Google Drive or create unnecessary output files when executed.
- **Choose an appropriate GPU.** A less powerful GPU may be sufficient for short experiments and can reduce the compute-unit consumption rate.



Problems with Colab?

- General advice: **even if this year we have Google Colab Pro licenses, please, use Colab resources judiciously**.
  - Otherwise, RAM problems may arise or you may be temporarily blocked/restricted from using the GPUs.

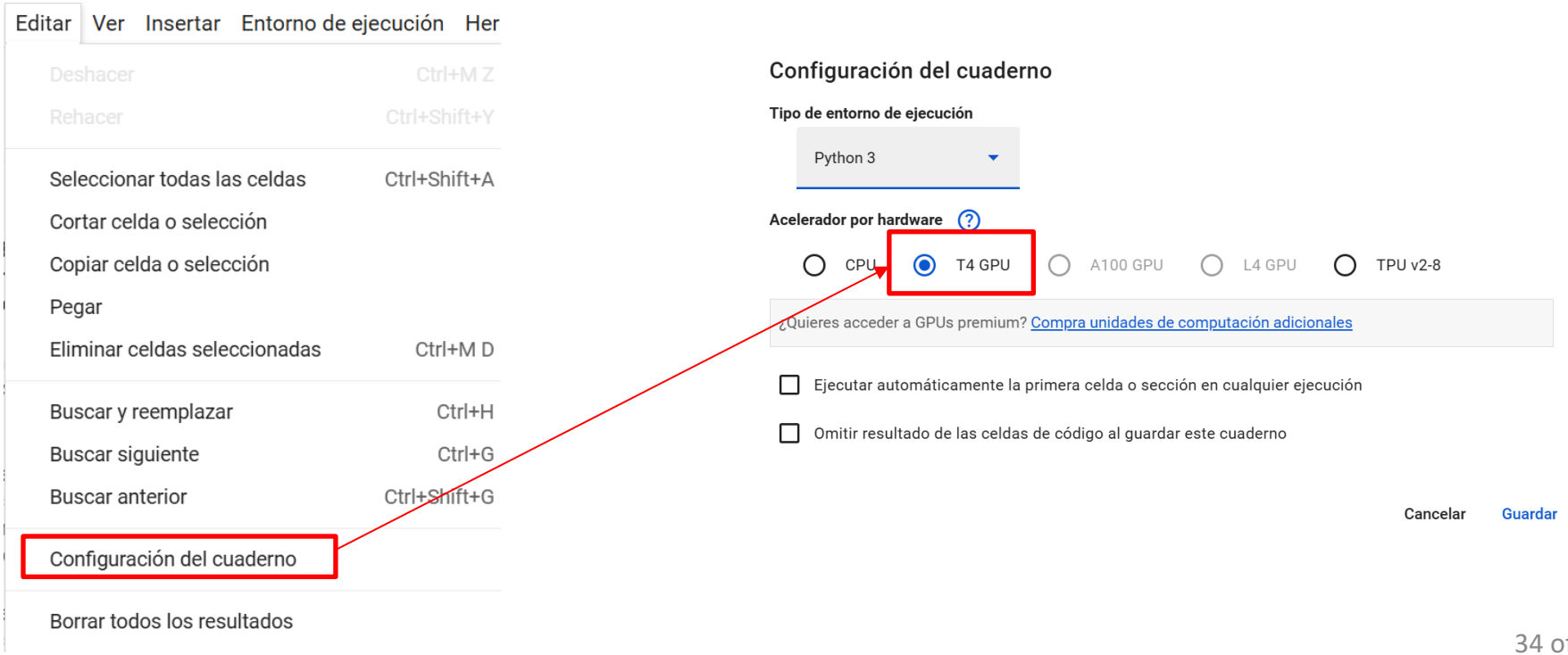
- If you employ good coding practices, and reasonable experiments are carried out in an orderly manner, there should be no problem.

First steps

- Bird classification
  - https://www.kaggle.com/code/jhoward/is-it-a-bird-creating-a-model-from-your-own-data (you will need internet access to run this Notebook → activate *SMS account verification*)
  - Presentation by Jeremy Howard: https://youtu.be/8SF_h3xF3cE?list=PLfYUBJiXbdtSvpQjSnJJ_PmDQB_VyT5iU&t=2314
- Bear classification
  - https://github.com/fastai/fastbook/blob/master/02_production.ipynb
- Imagenette images classification
  - https://docs.fast.ai/tutorial.imagenette.html

First steps

- Be sure that you are using GPU acceleration!



Data loading and manipulation

- DataBlock and DataLoaders

Key questions we want to answer to convert our data into a *DataLoaders* object:

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

What kind of data are we working with?

Where can we get the examples from?

How can we have a validation set?

Where do we get the labels from?

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

The inputs for our model will be images (*ImageBlock*) and the outputs are categories (*CategoryBlock*), such as "*bird*" or "*forest*"

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

We retrieve the images using the *get_image_files* function, which returns a list with all the images in *path* (https://docs.fast.ai/data.transforms.html#get_image_files)

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

We split the data randomly between training and validation (20%). We set the random seed to partition the data always in the same way.

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

The labels (desired outputs) are obtained from the name of the parent directory of each file (i.e. the name of the folder they are in) (https://docs.fast.ai/data.transforms.html#parent_label)

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

On each example (*item*) a series of transformations is applied: *resize* to 192x192 pixels, and "*squishing*" (as opposed to "*cropping*")

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

An interesting transformation is *RandomResizedCrop(size,min_scale)*. It randomly selects a crop whose area is at least *min_scale* times the original image area and resizes it to *size*.

https://github.com/fastai/fastbook/blob/master/02_production.ipynb
https://docs.fast.ai/vision.augment.html#randomresizedcrop

Data loading and manipulation

- DataBlock and DataLoaders

```
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=[Resize(192, method='squish')]
).dataloaders(path, bs=32)
```

What do you think is faster: to train with larger or smaller batch sizes? 🤔

All this process will be performed on the images/folders in *path*. And the images will be loaded in batches (*bs*) of 32.

You could also specify **`num_workers=os.cpu_count()`**
Number of CPU parallel subprocesses to prepare data before sending it to GPU!

DataLoaders documentation:
https://docs.fast.ai/data.load.html

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html

Data loading and manipulation

- Data normalization
  - One of the most common and recommended data transformations.
  - Inside the datablock:

```
batch_tfms= Normalize.from_stats(*imagenet_stats)
batch_tfms= Normalize()
batch_tfms= Normalize.from_stats(mean,std)
```

Note: "when using a pretrained model through *vision_learner*, the fastai library automatically adds the proper Normalize transform; the model has been pretrained with certain statistics in Normalize (usually coming from the ImageNet dataset), so the library can fill those in for you. Note that this only applies with pretrained models" (https://github.com/fastai/fastbook/blob/master/07_sizing_and_tta.ipynb)

Data loading and manipulation

```
pets = DataBlock(blocks=(ImageBlock, CategoryBlock),
                 get_items=get_image_files,
                 splitter=RandomSplitter(),
                 get_y=Pipeline([attrgetter("name"), RegexLabeller(pat = r'^(.*)_\d+.jpg$')]),
                 item_tfms=Resize(128),
                 batch_tfms=aug_transforms())
```

https://docs.fast.ai/tutorial.datablock.html

Standard set of augmentations that generally work pretty well.
https://github.com/fastai/fastbook/blob/master/02_production.ipynb

1) `item_tfms` is applied to each individual image before it is copied to the GPU. It resizes all images to same size.

2) `batch_tfms` is applied to a batch all at once on the GPU (it's fast).

https://github.com/fastai/fastbook/blob/master/05_pet_breeds.ipynb

Data loading and manipulation

- DataBlock and DataLoaders
  - There are different DataLoaders depending on the type of data you want to deal with and the problem you face:
    - ImageDataLoaders
    - SegmentationDataLoaders
    - LMDataLoader
    - TextDataLoader
    - TabularDataLoaders

Data block tutorial:
https://docs.fast.ai/tutorial.datablock.html
DataLoaders documentation:
https://docs.fast.ai/data.load.html

Creating a model from scratch

- Simple example:
  - Our input images are 32x32x3
  - We want a network with (in this order):
    - Convolutional layer with 5 7x7 filters, no padding and stride=1. ReLU activation function.
    - 2x2 MaxPooling.
    - Fully-connected layer with 15 neurons and Softmax activation function (with 15 output classes).

Creating a model from scratch

```
simpleNet = sequential(
    nn.Conv2d(in_channels=3,out_channels=5,kernel_size=(7,7)),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=(2,2)),
    nn.Flatten(),
    nn.Linear(in_features=845, out_features=15),
    nn.Softmax()
)
```

Here we have a volume of 26x26x5

Here we have a volume of 13x13x5, i.e. 845 elements

Is this really necessary?? Carefully check the documentation.

Creating a model from scratch

The *Learner* object includes the model (*simpleNet*), the data (*dls*) and the loss function (*loss_func*). We have everything to train our model.

```
learn = Learner(dls, simpleNet, loss_func=CrossEntropyLossFlat(), metrics=accuracy)
```

data

model

loss function

https://docs.fast.ai/losses.html

https://pytorch.org/docs/stable/nn.html#loss-functions

Metric for assessing performance

https://docs.fast.ai/metrics.html

Creating a model from scratch

```
learn.summary()
```

```
Sequential (Input shape: 32 x 3 x 32 x 32)
================================================================================
Layer (type)         Output Shape         Param #    Trainable
================================================================================
                     32 x 5 x 26 x 26
Conv2d                                    740        True
ReLU
________________________________________________________________________________
                     32 x 5 x 13 x 13
MaxPool2d
________________________________________________________________________________
                     32 x 845
Flatten
________________________________________________________________________________
                     32 x 15
Linear                                    12690      True
Softmax
________________________________________________________________________________

Total params: 13,430
Total trainable params: 13,430
Total non-trainable params: 0
```

`learn.summary()` allows you to verify that you have built the architecture correctly

5 filters with 7x7x3+1 (bias) parameters to learn

845*15 + 15 (bias) parameters to learn.

Note that **most of the weights are in the fully-connected**! Be careful not to include too many layers of this type when you create your own architecture!

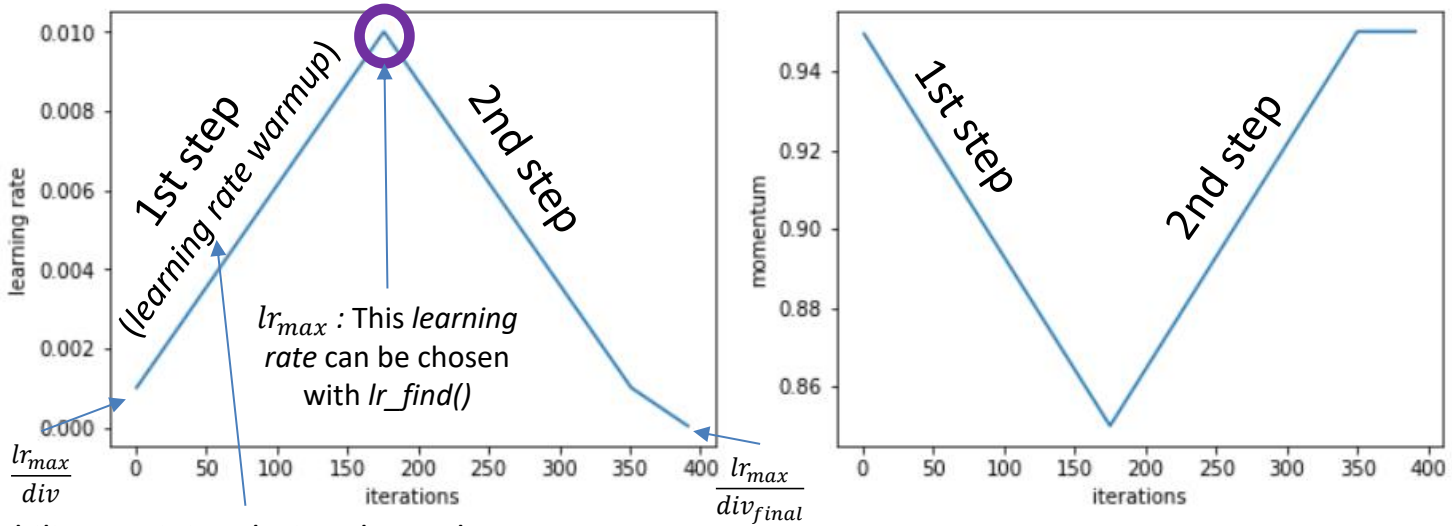
Batch size

Training

- learn.fit(n_epoch)
  - Trains the model for a certain number of epochs
- learn.fit_one_cycle(n_epoch)
  - Trains the model for a certain number of epochs using the *1cycle policy* of Leslie N. Smith (https://arxiv.org/abs/1708.07120)

Training

- learn.fit_one_cycle(n_epoch)
  - Trains the model for a certain number of epochs using the *1cycle policy* of Leslie N. Smith (https://arxiv.org/abs/1708.07120)



This combined strategy allows you to train much faster (*super-convergence*)

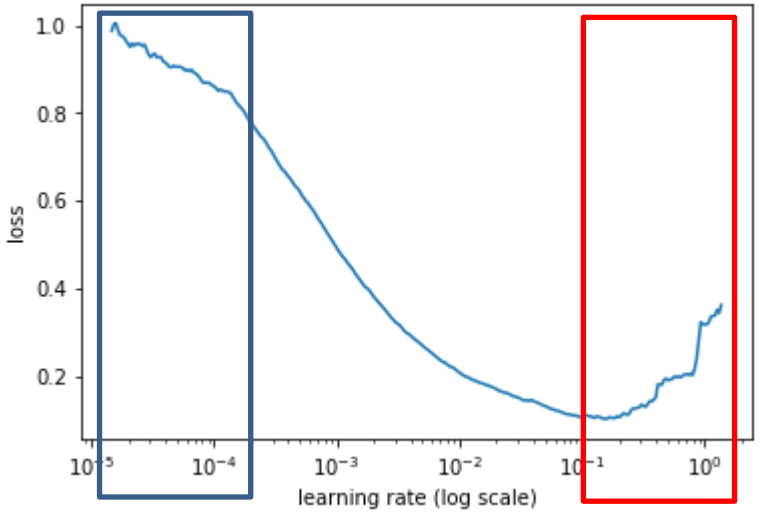
Stabilizes training during the early optimization phase when using large batch sizes (see Goyal 2017)

https://www.youtube.com/watch?v=CJKnDu2dxOE&t=7203s

Note: it's not 1 cycle per epoch, but 1 cycle for the whole training! If you want multiple cycles, check the cyclical learning rates paper.

Training

- learn.fit_one_cycle(n_epoch)
  - The maximum *learning rate* used in the *1cycle policy* is chosen with the *Learning Rate Finder*: *learner.lr_find()*



It performs a short training run over a fixed number of mini-batch iterations (100 by default), progressively increasing the learning rate.

It plots the training loss against the learning rate, helping us to choose a *learning rate* not **too big** or **too small**. We want to choose a *learning rate* as large as possible (without making the training diverge) to train/optimize as fast as possible.

https://sgugger.github.io/how-do-you-find-a-good-learning-rate.html

Training

- Multiple calls to *fit* or *fit_one_cycle*
  - like Keras, if these functions are called several times, we'd be training the model incrementally from the point/weights obtained from the previous call.
- Interesting possibility (with pre-trained models): *discriminative learning rates*



If you start with an uninitialized network, that you want to train from scratch, would it make sense to use it?

  - In the training function, use *slice()* to indicate the *learning rate*.
    - Example: `learn.fit_one_cycle(3, lr_max=slice(1e-5, 1e-3))`

    The head will train with 1e-3 and in previous layers will use smaller learning rates (with 3 layer groups, we'd use, as lr_max, 1e-5 in the first layer group and 1e-4 in the second one).

Training

- You can define, initialize and use different optimizers
  - https://docs.fast.ai/optimizer.html
  - Example using Adagrad (https://pytorch.org/docs/stable/generated/torch.optim.Adagrad.html)

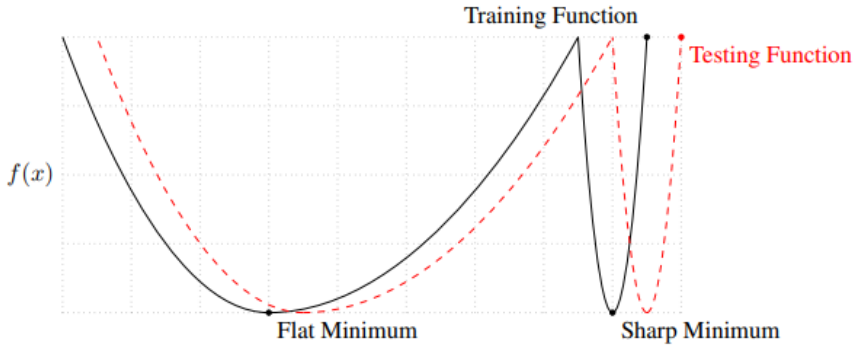
```
learn = Learner(dls, yourNetwork, metrics=accuracy,
        loss_func=LabelSmoothingCrossEntropyFlat(),
                        opt_func=partial(OptimWrapper,
                              opt=torch.optim.Adagrad,
                              lr=0.0001,lr_decay=0.01,
                                    weight_decay=0.1))
```

Training

- Pay attention to the relationship between different parameters: batch size (*bs*), learning rate (*lr*), weight decay (*wd*), momentum
  - small *lrs* tend to favor overfitting → large values should be used (but avoiding too large values that may prevent from converging).
  - small *bs* regularizes (it appears to locate minima with better generalization properties) → in small models perhaps better use big *bs*.
  - *wd* should be set to small values (the larger the value, the more regularization (higher *large weights* penalization)), otherwise it will be difficult to fit the data.
  - if we have a large *lr* and also a high momentum we run the risk of not converging.

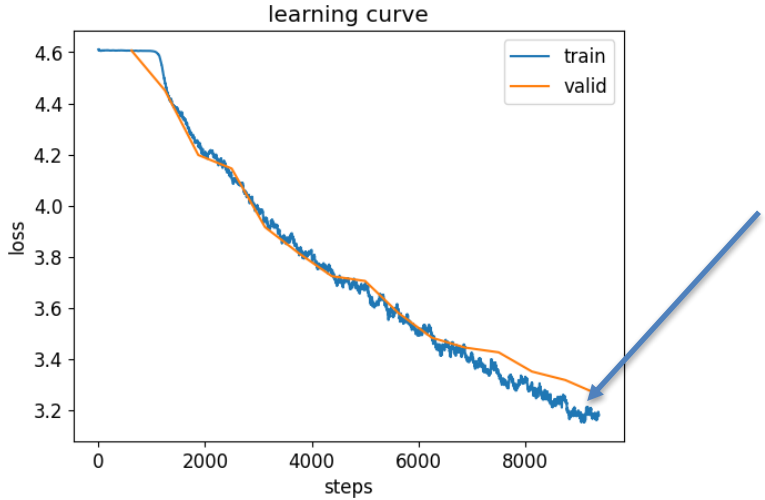
Training

- Sharp vs Flat Minima / Optimization of our loss vs Generalization ability
  - https://www.inference.vc/sharp-vs-flat-minima-are-still-a-mystery-to-me/
  - https://towardsdatascience.com/what-can-flatness-teach-us-understanding-generalisation-in-deep-neural-networks-a7d66f69cb5c
  - Zhou et al. (2020). Towards theoretically understanding why SGD generalizes better than Adam in deep learning. *Advances in Neural Information Processing Systems*, *33*, 21285-21296.
  - Hochreiter, S., & Schmidhuber, J. (1997). Flat minima. *Neural computation*, 9(1), 1-42.
  - Keskar et al. (2016). On large-batch training for deep learning: Generalization gap and sharp minima. arXiv preprint arXiv:1609.04836.



Training

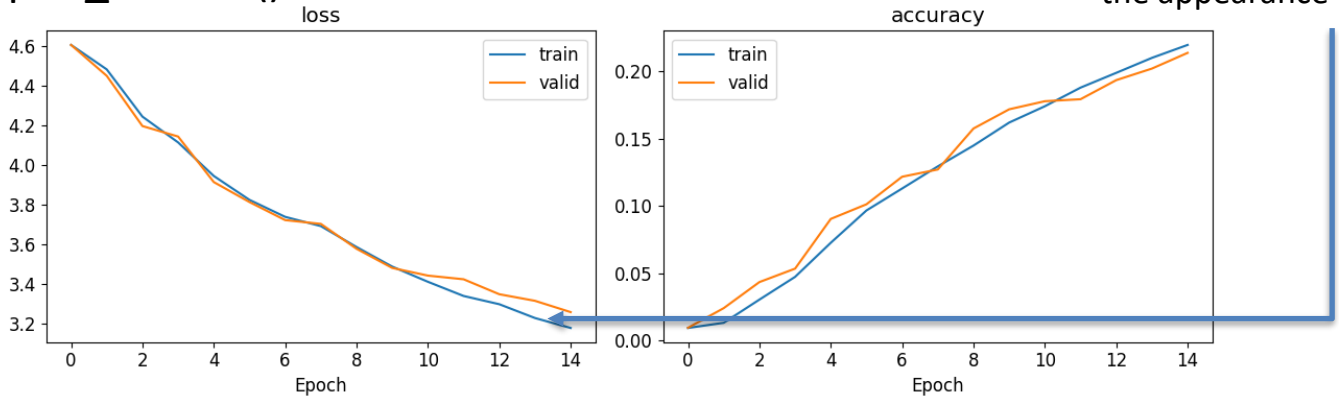
learn.recorder.plot_loss()



The “sawtooth” fluctuations reflect mini-batch variability and parameter updates during optimization.

Training loss is tracked per iteration (typically smoothed using an exponential moving average), whereas **validation loss is computed once per epoch**. These differences in sampling and smoothing affect the appearance of the curves.

learn.recorder.plot_metrics()



Prediction

- Once we have trained our model we can:
  - Perform prediction on a single example: `learn.predict(example)`
  - Perform prediction on a set of examples (test):

```
test_dl = learn.dls.test_dl(files_test,with_labels=True, device='cuda')
preds, targs = learn.get_preds(dl=test_dl)
```

Otherwise, it can be much slower!

- It is key to scale/normalize the test data following exactly the same protocol used in training (using the same mean and std)
  - This is done automatically for you by fastai, using `learn.get_preds` or `learn.dls.test_dl`.
    - https://forums.fast.ai/t/do-we-need-to-normalize-single-image-before-running-predict-function-on-it/44301/5
    - https://forums.fast.ai/t/99-accuracy-on-valid-data-1-accuracy-on-test-data-what-am-i-missing/80408/2

Interpretation of results

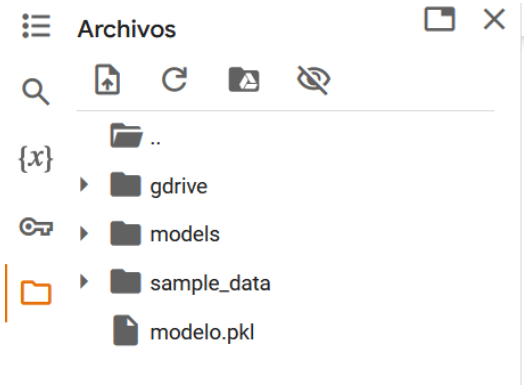
- `interp = ClassificationInterpretation.from_learner(learn)`
- `interp.plot_confusion_matrix(figsize=(12, 12),title='Title')`
- `interp.most_confused(min_val=10)`
- `interp.plot_top_losses(10, nrows=2, figsize=(32,4))`

And many other possibilities in https://docs.fast.ai/interpret.html

Saving and loading models

```
learn.export('modelo.pkl')
```

To know where it was saved: `learn.path`
Or check the folders browser:




```
from fastai.vision.all import load_learner
loaded_learn = load_learner('modelo.pkl')
```

Everything will be in the .pkl file within the learn.path folder. If you deploy your model on a different machine, this is the file you'll need to copy.

Once you have loaded the model, you can perform inference with it:

```
loaded_learn.predict(files_test[0])
```

Transfer Learning: Loading an existing model

- fastai integrates numerous trained state-of-the-art models:
  - https://timm.fast.ai/
  - https://github.com/huggingface/pytorch-image-models
- **Idea**: take something that already works well for a similar problem and reuse it in another problem (*transfer learning*).

Transfer Learning: Loading an existing model

```
model = fastai.vision.models.resnet18
learn = vision_learner(dls, model)
learn.summary()
```

We load the pre-trained model

We use *vision_learner* (https://docs.fast.ai/vision.learner.html): "All the functions necessary to build *Learner* suitable for transfer learning in computer vision"

We explore the architecture, as well as its trainable parameters

cnn_learner: Deprecated name for *vision_learner* – **do not use it**

Transfer Learning: Loading an existing model

- Does the above contain the weights?
  - By default, yes (*pretrained=True*). See https://docs.fast.ai/vision.learner.html
- How to modify the last layers of a pre-trained model?

```
custom_head = nn.Sequential(
    nn.AvgPool2d((7,7)),
    nn.Flatten(),
    nn.Linear(512, 200))

learn = vision_learner(dls, resnet18, custom_head=custom_head)
```

Fine-tuning

- learn.fine_tune(epochs, freeze_epochs)

  1) Trains the added layers (*head*) for one epoch (by default, freeze_epochs=1), with all other layers “frozen”.

  2) “Unfreezes” all layers, and trains them for the indicated number of epochs.

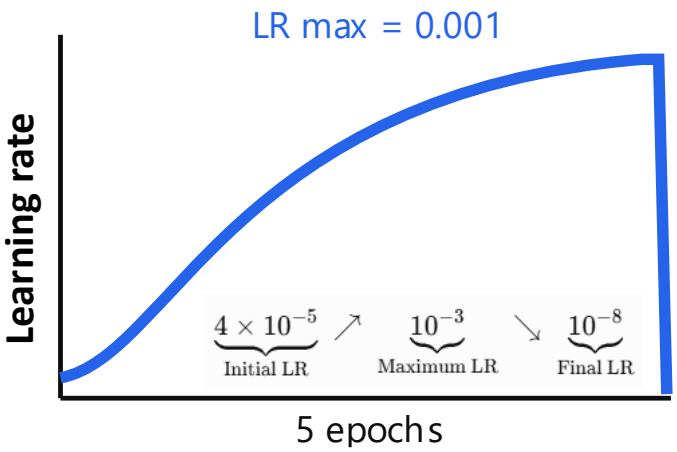
fine_tune() internally uses fit_one_cycle() with discriminative learning rates in the second phase.

See https://github.com/fastai/fastai/blob/master/fastai/callback/schedule.py#L161

Fine-tuning

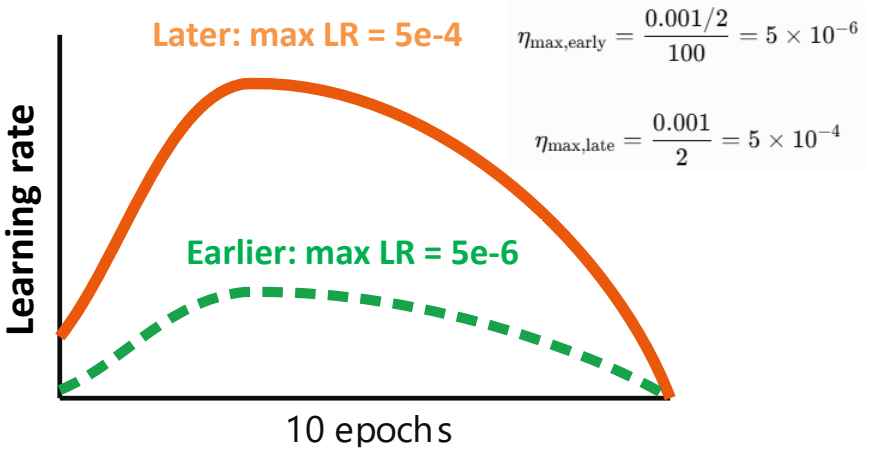
```
learn.fine_tune(epochs=10, base_lr=0.001, freeze_epochs=5)
```

Phase 1



Backbone: frozen (BatchNorm may be an exception).
Classification head: trainable.
LR increases for 99% of the training steps.
LR decreases for the remaining 1%.

Phase 2



Backbone: unfrozen.
Classification head: trainable.
Discriminative LR: Lower in earlier layers, higher in later layers.
LR increases for 30% of the training steps (3 epochs).
LR decreases for the remaining 70% (7 epochs).

Schematic representation (not to scale). Learning rates are updated after every training batch in both phases.

Fine-tuning

- If you want to “freeze” part of a model, so no changes happen to these weights (i.e. they’re not trained):

```
# We freeze all weights in the model
for param in model.parameters():
    param.requires_grad = False
```

https://stackoverflow.com/questions/51748138/pytorch-how-to-set-requires-grad-false

- You can also use **`learner.freeze()`**, **`learner.unfreeze()`**, **`learner.freeze_to()`**. See https://docs.fast.ai/learner.html and https://www.kaggle.com/code/danielliao/understanding-learner-freeze-to

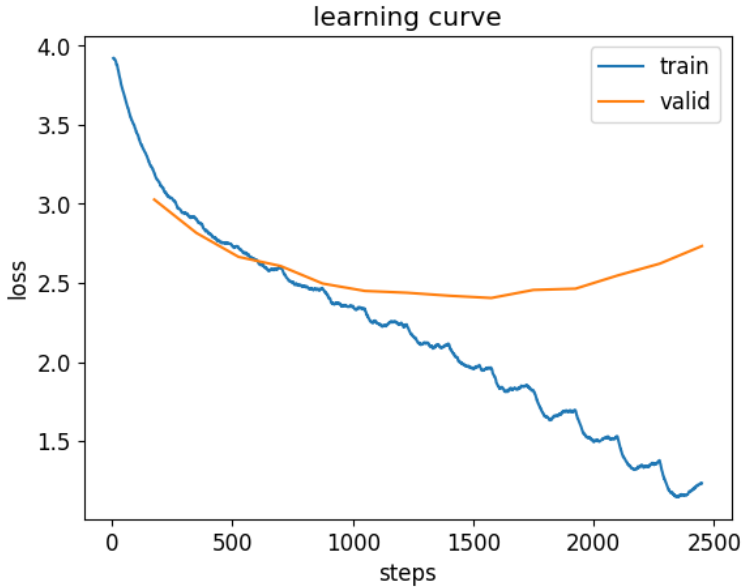
Fine-tuning

- If you use **`pretrained=False`** in **`vision_learner`** all layers will be trainable.

- If you use **`pretrained=True`**, all layers will be non trainable (except your new header and the batchnorm layers).
  - If you want batchnorm layers to be non trainable: **`train_bn=False`**
  - To fully freeze batchnorm layers in the pretrained backbone, use **`train_bn=False`** to freeze their trainable parameters and **`cbs=BnFreeze`** to prevent updates to their running statistics.

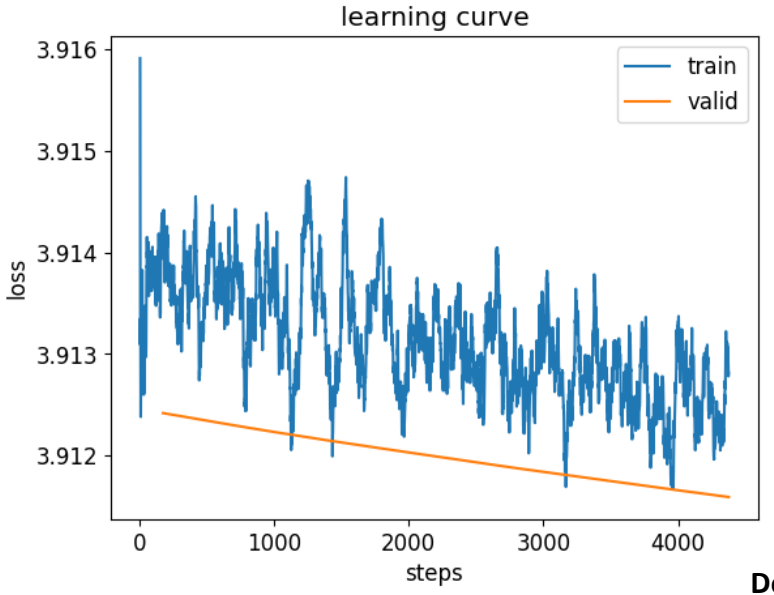
Be careful and pay attention to the details

```
learn.opt_func=partial(OptimWrapper,
opt=optim.SGD(learn.parameters(),
lr=0.0001, momentum=0.1))
```



fastai did not raise any errors or warnings. It simply ignored the call and used its default optimizer (Adam) under the hood.

```
learn = Learner(dls, BaseNet,
metrics=[accuracy, F1Score(average='macro')],
opt_func=partial(SGD, lr=0.0001, mom=0.1))
```



Double-check hyperparameter values!

```
print("Learner LR:", learn.lr)
print("Optimizer LR:", learn.opt.hypers[0]['lr'])
print("Optimizer hyperparameters:", learn.opt.hypers)
print(learn.opt_func)
```

Be careful and pay attention to the details

```
learn.fit(2)

learn.summary()
```

['epoch', 'train_loss', 'valid_loss', 'accuracy', 'f1_score', 'time']
(1, 3)

```
learn.recorder.plot_metrics()
```

```
----> 2 learn.recorder.plot_metrics()
      3 learn.show_results()
      4
      5 test_dl = dls.test_dl(get_image_files(path/'test'), with_labels=True)

/tmp/ipython-input-869448483.py in plot_metrics(self, nrows, ncols, figsize, **kwargs)
     17     axs = [ax if i < n else ax.set_axis_off() for i, ax in enumerate(axs.flatten())][:n]
     18     for i, (name, ax) in enumerate(zip(names, [axs[0]] + axs)):
---> 19         ax.plot(metrics[:, i], color='#1f77b4' if i == 0 else '#ff7f0e', label='valid' if i > 0 else 'train')
     20         ax.set_title(name if i > 1 else 'losses')
     21         ax.legend(loc='best')

IndexError: index 3 is out of bounds for axis 1 with size 3
```

```
print(learn.recorder.metric_names)
import numpy as np
print(np.array(learn.recorder.values).shape)
```

```
learn.summary()

learn.fit(2)
```

['epoch', 'train_loss', 'valid_loss', 'accuracy', 'f1_score', 'time']
(2, 4)

```
learn.recorder.plot_metrics()
```

Why does this happen?

learn.summary() performs a forward pass and may modify the internal state of fastai's Recorder callback. When called after training, **it can overwrite the recorded metrics**, causing a mismatch between metric names and stored values.

Best practice:

Call learn.summary() before learn.fit().

Be careful and pay attention to the details



Some fastai functions, such as summary(), show_results(), and get_preds(), **may reset or modify the Recorder state, affecting the stored training history**.

Best practice: Plot the training metrics immediately after fitting the model, before calling these functions.

Common Runtime Errors in fastai / PyTorch

1. Batch size mismatch

```
fa_collate: at least two tensors in the batch are not the same size.
```

Cause: Images have different spatial dimensions.

Solution: Apply item_tfms=Resize(224) before batching.

2. CUDA device-side assert triggered

```
RuntimeError: CUDA error: device-side assert triggered
```

Possible cause: Invalid class indices or an incorrect number of output logits for classification.

Solution: Check the output dimensions, target labels, and loss function. Fix the problem and restart the runtime if the CUDA error persists.

General advice

- **“It's only by practicing (and failing) a lot that you will get an intuition of how to train a model.”**
- Do not hesitate in consulting the online help directly in the Notebook:

  ??function `??learn.fine_tune`

  doc(function) `doc(learn.fine_tune)`

Computer Vision: Assignment 3
Deep Learning for Computer Vision

Pablo Mesejo and Siham Tabik
Universidad de Granada
Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA

